

Chapter 1

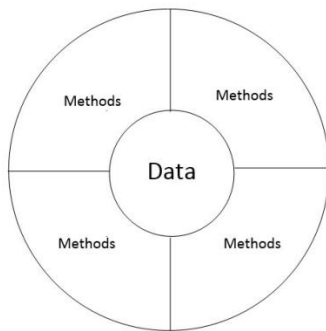
Fundamentals of Object-Oriented Programming

1.1 INTRODUCTION

Object-Oriented Programming (OOP) is an approach to program organization and development. It is a new way of organizing and developing programs

1.2 OBJECT-ORIENTED PARADIGM

The major objective of object-oriented approach is to eliminate some of the flaw encountered in the procedural approach. OOP treats data as a critical element in the program development and does not allow it to flow freely around the system. It ties data more closely to the functions that operate on it and protects it from unintentional modification by other functions. OOP allows us to decompose a problem into a number of entities called **Objects** and then build data and functions around these entities. The combination of data and methods make up an object (see below figure)



The data of an object can be accessed only by the methods associated with that object. However, methods of one object can access the methods of other objects. Some of the features of object-oriented paradigm are:

- Emphasis is on data rather than procedure
- Programs are divided into objects
- Data is hidden and cannot be accessed by external functions
- Objects communicate with each other through methods
- New data and methods can be easily added whenever necessary

1.3 BASIC CONCEPTS OF OOP

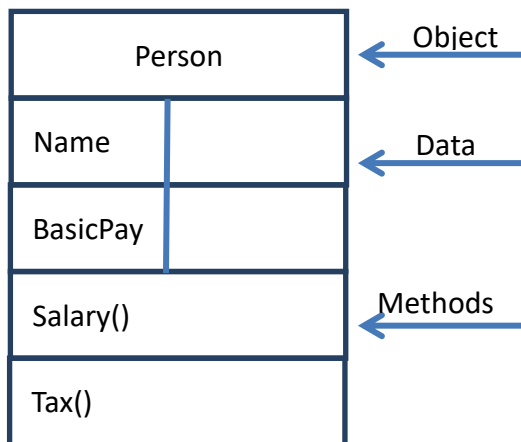
The general concepts of OOP which form the heart of Java language

1. Objects and Classes
2. Data abstraction and Encapsulation

3. Inheritance
4. Polymorphism
5. Compile Time and Runtime Mechanisms
6. Dynamic Binding
7. Message Communication

Object and Classes

Objects are the basic runtime entities. It represents a person, a place, a bank account, a table of data or any item. They may also represent user-defined data types. When a program is executed, the objects interact by sending messages to one another. For example, 'customer' and 'account' are 2 objects in a banking program, then the customer object may send a message to the account object requesting for the balance. Each object contains data and code. The below figure shows a notation that is popularly used to represent an object in object-oriented analysis and design



The entire set of data and code of an object can be made a user-defined data type using the concept of a class. A class may be thought of as a 'data type' and an object as a 'variable' of that data type. Once a class has been defined, we can create any number of objects belonging to that class. Each object is associated with the data of type class with which they are created. A class is thus a collection of objects of similar type. For example, mango, apple, and orange are members of the class fruit

Data Abstraction and Encapsulation

The wrapping up of data and methods into a single unit (called class) is known as **encapsulation**. The data is not accessible to the outside world and only those methods, which are wrapped in the class, can access it. These methods provide the

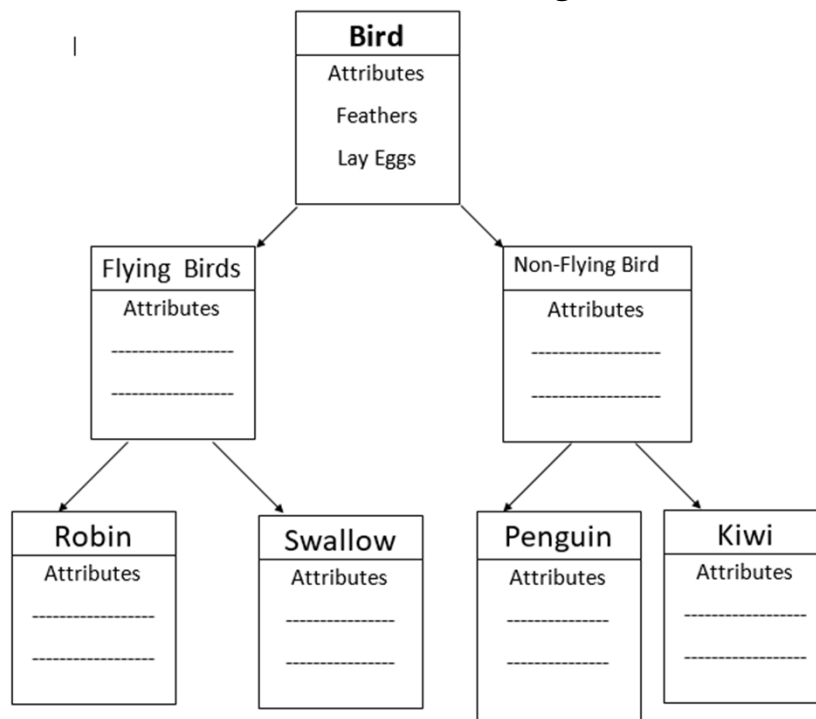
interface between the object's data and the program. This insulation of the data from direct access by the program is called **data hiding**. Encapsulation makes it possible for objects to be treated like 'black boxes', each performing a specific task without any concern for internal implementation (see below figure)



Abstraction refers to the act of representing essential features without including the background details or explanations. Example, mobile phones

Inheritance

Inheritance is the process by which objects of one class acquire the properties of another class. Inheritance supports the concept of hierarchical classification. For example, the bird robin is a part of the class flying bird, which is again a part of the class bird. It is illustrated in below figure.

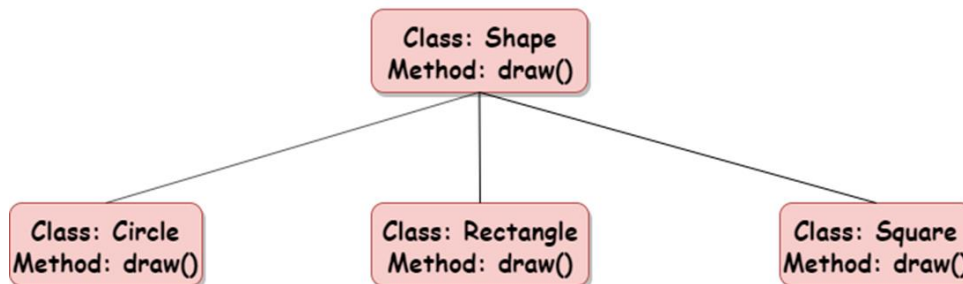


In OOP, the concept of inheritance provides the idea of reusability. This means that we can add additional features to an existing class without modifying it. This is

possible by deriving a new class from the existing one. The new class will have the combined features of both the classes

Polymorphism

Polymorphism is another important OOP concept. Polymorphism means the ability to take more than one form. For example, consider the operation of addition. For 2 numbers, the operation will generate a sum. If the operands are strings, then the operation would produce a third string by concatenation. The below figure illustrates that a single function name can be used to handle different number and different types of arguments



Compile Time and Runtime Mechanisms

Compile time may refer to the following

- The operations performed by the compiler
- The pre-runtime assessment of the programs behavior

Runtime is the time period when a syntactically correct program is actually executed in the computer system

Dynamic Binding

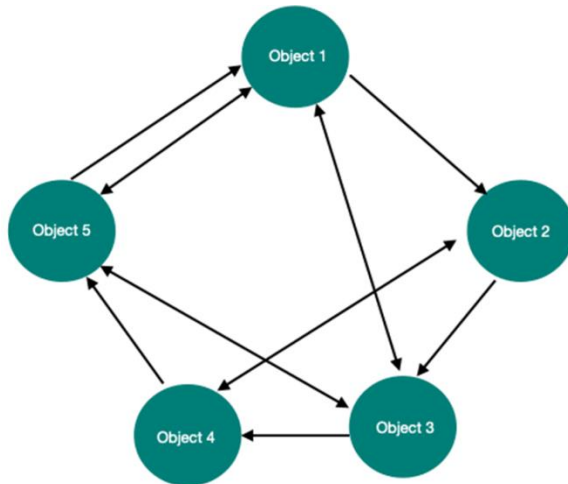
Binding refers to the linking of a procedure call to the code to be executed in response to the call. Dynamic binding means that the code associated with a given procedure call is not known until the time of the call at runtime. It is associated with polymorphism and inheritance

Message Communication

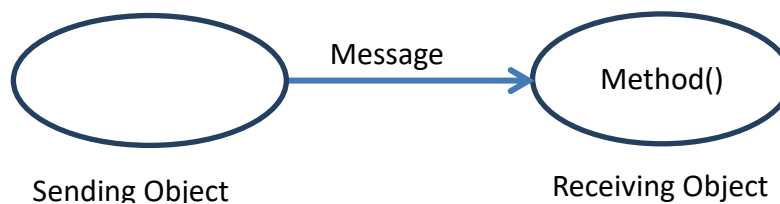
An OOP consists of a set of objects that communicate with each other. The process of programming in an object-oriented language involves the following basic steps

1. Creating classes that define objects and their behavior
2. Creating objects from class definitions
3. Establishing communication among objects

Objects communicate with one another by sending and receiving information much the same way as people pass messages to one another as shown in below figure.



A message for an object is a request for execution of a procedure, and therefore will invoke a method in the receiving object that generates the desired result, as shown in below figure



Message passing involves specifying the name of the object, the name of the method and the information to be sent. For example: `Employee.Salary(name);` Objects have a life cycle. They can be created and destroyed. Communication with an object is feasible as long as it is alive

1.4 BENEFITS OF OOP

OOP offers several benefits to both the program designer and the user. The principal advantages are:

- Through inheritance, we can eliminate redundant code and extend the use of existing classes.
- It saves development time and gives higher productivity.
- Data hiding helps the programmer to build secure programs.
- It is possible to have multiple objects to coexist without any interference.
- It is easy to partition the work in a project based on objects.
- Easily upgraded from small to large systems.

- Message passing techniques for communication between objects make the interface descriptions with external systems much simpler.
- Software complexity can be easily managed

1.5 Applications of OOP

The promising areas for application of OOP include:

- Real-time systems
- Simulation and modeling
- Object-oriented databases
- Hypertext, hypermedia and expert text
- AI and expert systems
- Neural networks and parallel programming
- Decision support and office automation systems
- CIM/CAD system

Chapter 2

Introduction to Java

2.1 JAVA HISTORY

Java is a general purpose, object-oriented programming language developed by sun Microsystems of USA in 1991. It was originally called as Oak by James Gosling. It is a simple language, portable and highly reliable. The Java team which included Patrick Naughton, James Gosling, Chris Warth, Ed Frank, and Mike Sheridan. It is a platform-neutral language

2.2 JAVA FEATURES

- Compiled and Interpreted
- Platform-Independent and Portable
- Object-Oriented
- Robust and Secure
- Distributed
- Familiar, Simple, and Small
- Multithreaded and Interactive

- High Performance
- Dynamic and Extensible
- Ease of Development
- Scalability and Performance
- Monitoring and Manageability
- Desktop Client

Compiled and Interpreted

A computer language is either compiled or interpreted. Java is a two-stage system. First, Java compiler translates source code into bytecode instructions. Bytecodes are not machine instructions and therefore, in the second stage, Java interpreter generates machine code that can be directly executed by the machine. Java is both a compiled and an interpreted language.

Platform Independent and Portable

Java programs can be easily moved from one computer system to another, anywhere and anytime. Changes and upgrade in operating systems, processors and system resources will not force any changes in Java programs. Java ensures portability in 2 ways. First, Java compiler generates bytecode instructions that can be implemented on any machine. Secondly, the size of the primitive data types are machine independent.

Object-Oriented

Java is a true object-oriented language. Almost everything in Java is an object. All program code and data reside within objects and classes. The object model in Java is simple and easy to extend.

Robust and Secure

Java is a robust language. It provides many safeguards to ensure reliable code. It has strict compile time and runtime checking for data types. It is a garbage collected language. Java incorporates the concept of exception handling which captures series errors and eliminates any risk of crashing the system. Java systems ensure that no viruses are communicated with an applet.

Distributed

Java is designed as a distributed language. It has the ability to share both data and programs.

Simple, Small and Familiar

Java is a simple and small language. Many features of C and C++ that are either redundant or sources of unreliable code are not part of Java. Java does not use pointer, preprocessor header files, goto statement and many others. It also eliminates operator overloading and multiple inheritance. Java is modelled on C and C++ languages. Java uses many constructs of C and C++. Java code looks like a C++ code. Java is a simplified version of C++

Multithreaded and Interactive

Multithreaded means handling multiple tasks simultaneously. Java supports multithreaded programs. We need not wait for the application to finish one task before beginning another. This improves the interactive performance of graphical applications

High Performance

Java architecture is designed to reduce overheads during runtime. Overall executions speed of Java programs is good

Dynamic and Extensible

Java is capable of dynamically linking in new class libraries, methods and objects. Java can also determine the type of class through a query, making it possible to either dynamically link or abort the program, depending on the response. Java programs support functions written in other languages such as C and C++. These functions are called native methods. Native methods are linked dynamically at runtime

Ease of Development

Java 2 Standard Edition (J2SE) 5.0 supports different features. These features reduce the work of the programmer by shifting the responsibilities to the compiler. The resulting source code is free from bugs because errors made by the compiler are less when compared to those made by programmers

Scalability and Performance

Improves the startup time and reduces the amount of memory used in runtime environment

Monitoring and Manageability

Java supports a number of APIs, such as JVM Monitoring and Management API to monitor and manage Java applications

Desktop Client

J2SE 5.0 provides enhanced features to meet the requirement and challenges of the Java desktop users

2.3 JAVA ENVIRONMENT

Java environment includes a large number of development tools and hundreds of classes and methods. The development tools are part of the system known as Java Development Kit (JDK) and the classes and methods are part of the Java Standard Library (JSL), also known as the Application Programming Interface (API)

Java Development Kit (JDK)

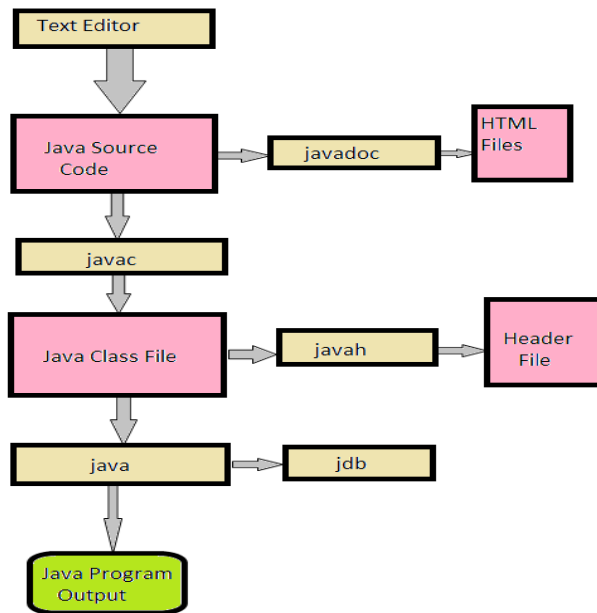
The JDK comes with a collection of tools that are used for developing and running Java programs. They include:

- appletviewer (for viewing java applets)
- javac (java compiler)
- java (java interpreter)
- javap (java disassembler)
- javah (for c header files)
- javadoc (for creating html documents)
- jdb (java debugger)

The below table lists these tools and their descriptions

Tool	Description
appletviewer	Enables to run Java applets
Java	It runs applets and applications
Javac	Translates Java source code to bytecode files
Javadoc	Creates HTML-format documentation from Java source code files
Javah	Produces header files for use with native methods
Javap	Enables to convert bytecode files into a program description
Jdb	Helps to find errors in our programs

The way these tools are applied to build and run application programs is illustrated in below figure



To create a Java program, we need to create a source code file using a text editor. The source code is then compiled using the Java compiler **javac** and executed using the Java interpreter **java**. The Java debugger **jdb** is used to find errors in the source code. A compiled Java program can be converted into a source code with the help of Java disassembler **javap**

Application Programming Interface

The Java Standard Library (or API) includes hundreds of classes and methods grouped into several functional packages. Most commonly used packages are:

- **Language Support Package:** A collection of classes and methods required for implementing basic features of Java
- **Utilities Package:** A collection of classes to provide utility functions such as date and time functions
- **Input/Output Packages:** A collection of classes required for input/output manipulation
- **Networking Package:** A collection of classes for communication with other computers via Internet
- **AWT Package:** The Abstract Window Tool Kit package contains classes that implements platform-independent graphical user interface
- **Applet Package:** This includes a set of classes that allows us to create Java applets

Java Runtime Environment (JRE)

The JRE facilitates the execution of programs developed in Java. It primarily comprises of the following:

- **Java Virtual Machine (JVM):** It is a program that interprets the intermediate Java byte code and generates the desired output
- **Runtime class libraries:** These are a set of core class libraries that are required for the execution of Java programs
- **User interface toolkits:** AWT and Swing are examples of toolkits that support varied input methods for the users to interact with the application program
- **Deployment technologies:** JRE comprises the following key deployment technologies:
 - **Java plug-in:** Enables the execution of a Java applet on the browser
 - **Java Web Start:** enables remote-deployment of an application

2.4 SIMPLE JAVA PROGRAM

```
class SampleOne
{
public static void main(String args[])
{
System.out.println("Java is better than C++");
}
}
```

OUTPUT

Java is better than C++

Class Declaration

The first line `class SampleOne` declares a class. Java is a true object-oriented language and therefore, **everything** must be placed inside a class. **class** is a keyword and declares that a new class definition follows. **SampleOne** is a Java identifier that specifies the name of the class to be defined

Opening Brace

Every class definition in Java begins with an opening brace “{” and ends with a matching closing brace “}”. This is similar to C++ construct

The Main Line

The third line `public static void main(String args[])` defines a method named **main**. This is similar to the **main()** function in C/C++. Every Java application program must include the **main()** method. This is the starting point for

the interpreter to begin the execution of the program. This line contains other keywords **public**, **static** and **void**

- **public**: It is an access specifier that declares the main method as unprotected and therefore making it accessible to all other classes.
- **static**: The **main** must always be declared as **static** since the interpreter uses this method before any objects are created
- **void**: It states that the **main** method does not return any value

String args[] declares a parameter named **args**, which contains an array of objects of the class type **String**

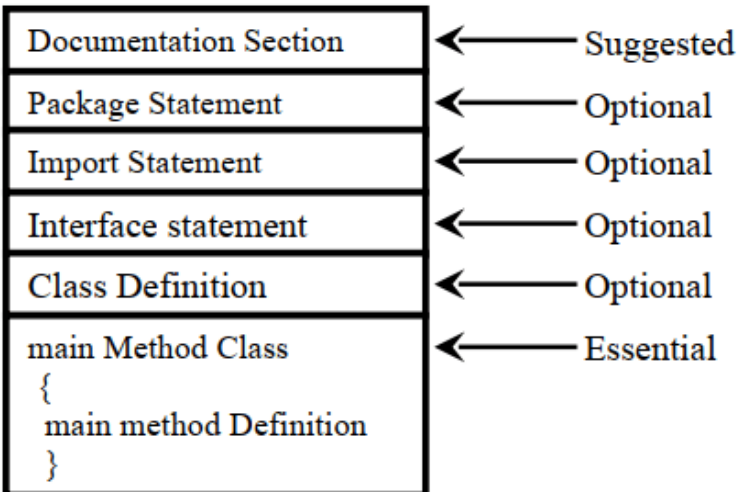
The Output Line

The only executable statement in the program is: **System.out.println("Java is better than C++");**

This is similar to the **printf()** statement of C or **cout<<** construct of C++. Since Java is a true object-oriented language, every method must be part of an object. The **println** method is a member of the **out** object, which is a static data member of **System** class. This line prints the string **Java is better than C++**. The method **println** always appends a newline character to the end of the string. This means that any subsequent output will start on a new line. Every Java statement must end with a semicolon

2.5 JAVA PROGRAM STRUCTURE

A Java program may contain many classes of which only one class defines a main method. Classes contain data members and methods. Methods may contain data type declaration and executable statements. To write a Java program, we first define classes and then put them together. A Java program may contain one or more sections



Documentation Section

A documentation section comprises a set of comment lines giving the name of the program, the author and other details. Comments must explain **why** and **what** of classes and **how** of algorithms. Java permits both the single line and multi-line comments. The single line comments begin with `//` and end at the end of the line. For longer comments, we can create long multi-line comments by starting with a `/*` and ending with a `*/`. Java also uses a third style of comment `/**....*/` known as documentation comment

Package Statement

The first statement allowed in a Java file is a **package** statement. This statement declares a **package** name and informs the compiler that the classes defined here belongs to this package.

Example: `package student;`

The package statement is optional

Import Statements

This is similar to the **#include** statement in C.

Example: `import student.test;`

This statement instructs the interpreter to load the **test** class contained in the package **student**. Using import statements, we can have access to classes that are part of other named packages

Interface Statements

An interface is like a class but includes a group of method declaration. This is also an optional section and is used only when we wish to implement the multiple inheritance features in the program

Class Definitions

A Java program may contain multiple class definitions. Classes are the primary and essential elements of a Java program. The number of classes used depends on the complexity of the problem

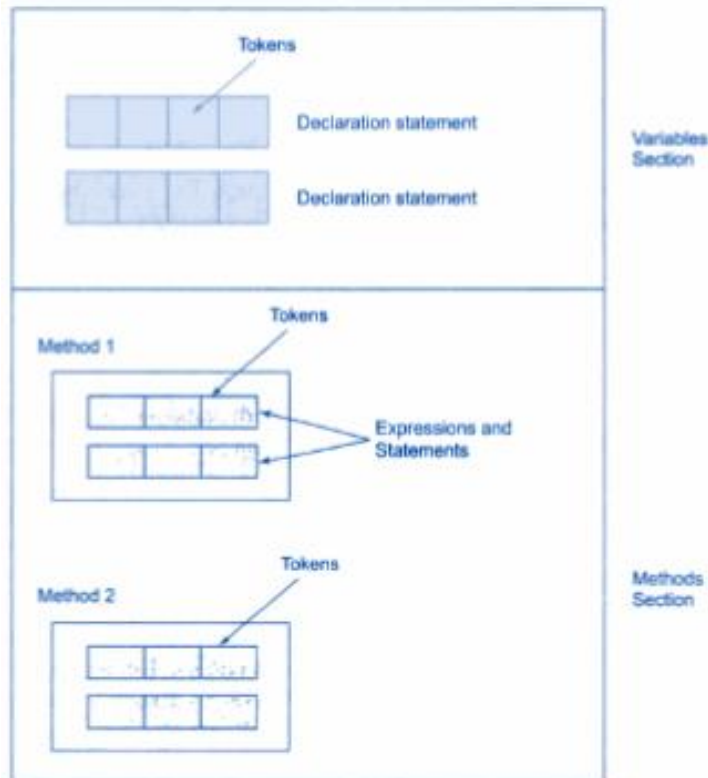
Main Method Class

Since every Java program requires a **main** method as its starting point, this class is the essential part of a Java program. A simple Java program may contain only this part. The **main** method creates objects of various classes and establishes communications between them. On reaching the end of **main**, the program terminates and the control passed back to the operating system

2.6 JAVA TOKEN

A Java program is basically a collection of classes. A class is defined by a set of declaration statements and methods containing executable statements (see the below figure). Most statements contain expressions, which describe the actions carried out on data. Smallest individual units in a program are known as **tokens**. A Java program is a collection of tokens, comments and white spaces. Java language includes 5 types of tokens

- Reserved Keywords
- Identifiers
- Literals
- Operators
- Separators



Java Character Set

The smallest units of Java language are the characters used to write Java tokens. These characters are defined by the **Unicode** character set. The Unicode is a 16 bit character coding system and currently supports more than 34,000 defined characters derived from 24 languages from America, Europe, Middle East, Africa and Asia. Most of us use only the basic ASCII characters, which include letters, digits and punctuation marks, used in normal English

Keywords

Keywords are an essential part of a language definition. Java language has reserved 50 words as keywords. The below figure lists these keywords. These keywords are combined with operators and separators according to syntax. Since keywords have specific meaning in Java, we cannot use them as names for variables, classes, methods and so on. All keywords are to be written in lower case letters. Since Java is case sensitive, one can use these words as identifiers by changing one or more letters to upper case. However it is a bad practice and should be avoided

abstract	default	if	private	throw
assert	do	implements	protected	throws
boolean	double	import	public	transient
break	else	instanceof	return	true
byte	extends	int	short	try
case	false	interface	static	void
catch	final	long	strictfp	volatile
char	finally	native	super	while
class	float	new	switch	
const	for	null	synchronized	
continue	goto	package	this	

Identifiers

Identifiers are programmer designed tokens. They are used for naming classes, methods, variables, objects, labels, packages and interfaces in a program. Java identifiers follow the following rules:

1. They can have alphabets, digits and the underscore and dollar sign characters
2. They must not begin with a digit
3. Uppercase and lowercase letters are distinct
4. They can be of any length

Identifier must be meaningful, short enough to be quickly and easily types and long enough to be descriptive and easily read. Java developers have followed some naming conventions

- Names of all public methods and instance variables start with a leading lowercase letter. Examples: average, sum
- When more than one word is used in a name, the second and subsequent words are marked with a leading uppercase letters. Examples: dayTemperature, firstDayOfMonth, totalMarks
- All private and local variables use only lowercase letters combined with underscores. Examples: length, batch_strength
- All classes and interfaces start with a leading uppercase letter and each subsequent word with a leading uppercase letter. Example: Student, HelloJava, Vehicle, MotorCycle
- Variables that represent constant values use all uppercase letters and underscores between words. Example: TOTAL, F_MAX, PRINCIPAL_AMOUNT

It should be remembered that all these are conventions and not rules. We may follow our own conventions

Literals

Literals in Java are a sequence of characters (digits, letters and other characters) that represent constant values to be stored in variables. Java language specifies 5 major types of literals

1. Integer literals
2. Floating_point literals
3. Character literals
4. String literals
5. Boolean literals

Each of them has a type associated with it. The type describes how the values behave and how they are stored

Operators

An operator is a symbol that takes one or more arguments and operates on them to produce a result

Separators

Separators are symbols used to indicate where groups of code are divided and arranged. The below table lists separators and their functions

Name	Symbol	Use
parentheses	()	Used to enclose parameters in method definition. Used for defining precedence in expressions and surrounding cast types
braces	{ }	Used to contain the values of automatically initialized arrays and define a block of code for classes, methods and local scopes
brackets	[]	Used to declare array types
semicolon	;	Used to separate statements
comma	,	Used to separate identifiers
period	.	Used to separate package names from sub packages and classes

2.7 JAVA STATEMENTS

The statements in Java are like sentences in natural languages. A statement is an executable combination of tokens ending with a semicolon mark. Statements are usually executed in sequence in the order in which they appear. Java implements several types of statements which is described in below table.

Statement	Description
Empty Statement	These do nothing
Labelled Statement	Any statement may begin with a label. Such labels must not be keywords, already declared local variables or previously used labels
Expression Statement	Java has 7 types of Expression statements: Assignment, Pre-Increment, Pre-Decrement, Post-Increment, Post-Decrement, Method Call and Allocation Expression
Selection Statement	These select one of several control flows. There are 3 types of selection statements in Java: if, if-else, and switch
Iteration Statement	These specify how and when looping will take place. There are 3 types of iteration statements: while, do and for
Jump Statement	These pass control to the beginning or end of the current block or to a labeled statement. The 4 types of Jump statements are break, continue, return and throw
Synchronization Statement	These are used for handling issues with multithreading
Guarding Statement	These are used for safe handling of code that may cause exceptions (division by zero). These statements use the keywords try, catch and finally

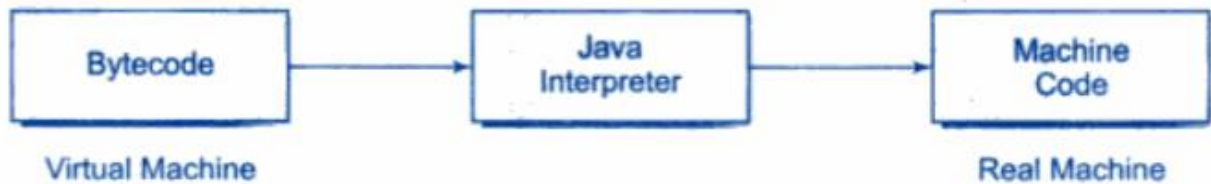
2.8 JAVA VIRTUAL MACHINE

All language compilers translate source code into machine code for a specific computer. Java compiler also does the same thing. Then how does Java achieve architecture neutrality? The answer is that the Java compiler produces an intermediate code known as **bytecode** for a machine that does not exist. This machine is called the **Java Virtual Machine** and it exists only inside the computer memory. The below figure illustrates the process of compiling a Java program into bytecode which is also referred to as virtual machine code

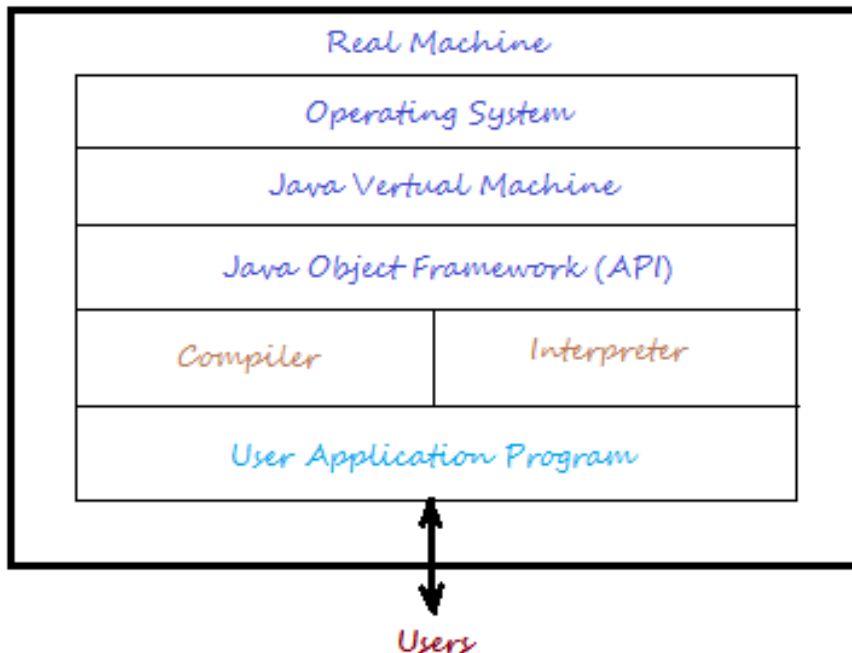


The virtual machine code is not machine specific. The machine specific code

(machine code) is generated by the Java interpreter by acting as an intermediary between the virtual machine and the machine as shown in below figure.



The below figure illustrates how Java works on a typical computer



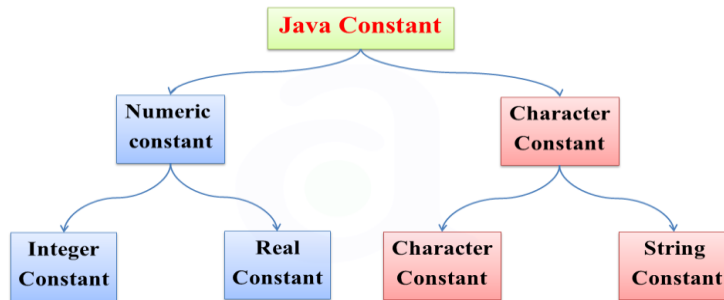
The Java object framework (Java API) acts as the intermediary between the user programs and the virtual machine which in turn acts as the intermediary between the operating system and the Java object framework

Chapter 3

Java Programming Basics

3.1 CONSTANTS

Constants in Java refer to fixed values that do not change during the execution of a program. Java supports several types of constants as illustrated in below figure



Integer Constants

An integer constant refers to a sequence of digits. There are 3 types of integers, namely, **decimal** integer, **octal** integer and **hexadecimal** integer

Decimal integers consist of a set of digits 0 through 9, preceded by an optional minus sign. Valid examples of decimal integer constants are: 123, -321, 0, 654321. Embedded spaces, commas, and non-digit characters are not permitted between digits. For example: 15 750, 20.000, \$1000 are illegal numbers.

An octal integer constant consists of any combination of digits from the set 0 through 7, with a leading 0. Some examples of octal integer are: 037, 0, 0435, 0551. A sequence of digits preceded by 0x or 0X is considered as hexadecimal integer (hex integer). They may also include alphabets A through F or a through f. A letter A through F represents the numbers 10 through 15. Some of the examples of valid hex integers are: 0X2, 0X9F, 0xbcd, 0x. We rarely use octal and hexadecimal numbers in programming.

Real Constants

Integer numbers are inadequate to represent quantities that vary continuously, such as distances, heights, temperatures, prices and so on. These quantities are represented by numbers containing fractional parts like 17.548. Such numbers are called **real (floating point)** constants. Further examples of real constants are: 0.0083, -0.75, 435.36.

These numbers are shown in **decimal notation**, having a whole number followed by a decimal point and the fractional part, which is an integer. It is possible that the number may not have digits before the decimal point or digits after the decimal point. That is, 215. , .95, -.71 are all valid numbers.

A real number may also be expressed in **exponential (or scientific)** notation. For example, the value 215.65 may be written as 2.1565e2 in exponential notation. e2 means multiply by 10^2 . The general form is:

mantissa e exponent

The **mantissa** is either a real number expressed in **decimal notation** or an integer. The **exponent** is an integer with an optional **plus** or **minus** sign. The letter

separating the mantissa and the exponent can be written in either lowercase or uppercase. Since the exponent causes the decimal point to “float”, this notation is said to represent a real number in floating point form. Examples of legal floating point constants are: 0.65e4, 12e-2, 1.5e+5, 3.18E3, -1.2E-1

Embedded white space is not allowed, in any numeric constant.

A floating point constant may thus comprise 4 parts:

- A whole number
- A decimal point
- A fractional part
- An exponent

Single Character Constants

A single character constant (or simply character constant) contains a single character enclosed within a pair of single quote marks. Examples of character constants are: ‘5’, ‘X’, ‘;’, ‘ ’

String Constants

A string constant is a sequence of characters enclosed within double quotes. The characters may be alphabets, digits, special characters and blank spaces. Examples are: “Hello Java”, “1997”, “WELLDONE”, “5+3”, “X”

Backslash Character Constants

Java supports some special backslash character constants that are used in output methods. A list of backslash character constants is given in below table. Each one of them represents one character, although they consist of 2 characters. These characters combinations are known as **escape sequences**

Constant	Meaning
\b	Backspace
\f	Form feed
\n	New line
\r	Carriage return
\t	Horizontal tab
\'	Single quote
\"	Double quote
\\	Backslash

3.2 VARIABLES

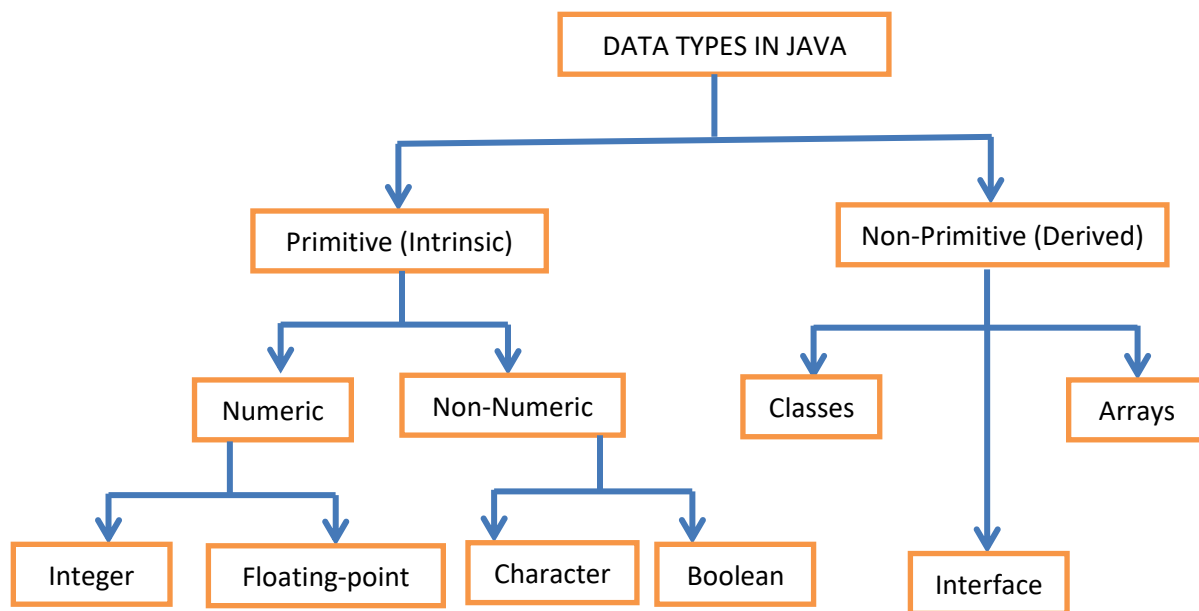
A variable is an identifier that denotes a storage location used to store a data value. A variable may take different values at different times during the execution of the program. A variable name may be chosen by the programmer in a meaningful way. Some examples of variables names are: avearge, height, total_height, classStrength

Variable names may consist of alphabets, digits, the underscore (_) and dollar characters, subject to the following conditions:

1. They must not begin with a digit
2. Uppercase and lowercase are distinct. Total is not name as total or TOTAL
3. It should not be a keyword
4. White space is not allowed
5. Variable names can be of any length

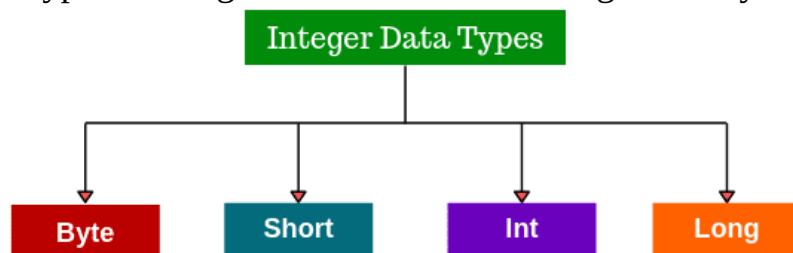
3.3 DATA TYPES

Every variable in Java has a data type. Data types specify the size and the type of values that can be stored. Java language is rich in data types. Data types in Java under various categories are shown in below figure.



Integer Types

Integer types can hold whole numbers such as 123, -96, 5639. The size of the values that can be stored depends on the integer data type we choose. Java supports 4 types of integers as shown in below figure. They are **byte**, **short**, **int** and **long**.



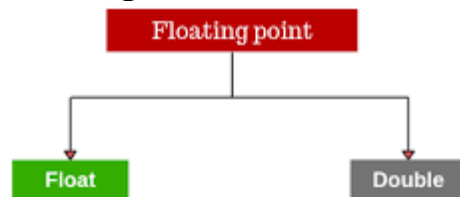
The below table shows the memory size and range of all four integer data types.

sno	Data type	Size(in bytes)	range
1	byte	1	+127 to -128
2	short	2	+32767 to -32767
3	int	4	+2147483647 to -2147483647
4	long	8	+9223372036854775807 to -9223372036854775808

It should be remembered that wider data types require more time for manipulation and therefore it is advisable to use smaller data types. For example, instead of storing a number like 50 in an **int** type variable, we must use a **byte** variable to handle this number. This will improve the speed of execution of the program. We can make **long** by appending the letter L or l at the end of the number. Example: 123L, 123l

Floating Point Types

Integer types can hold only whole numbers and therefore we use another type known as floating point type to hold numbers containing fractional part such as 27.59 and -1.375. There are 2 kinds of floating point storage in Java as shown in below figure



The **float** type values are **single precision number** while **double** types represent **double-precision numbers**. The below table gives the size and range of these two types.

Data type	Size	Range
Float	4 bytes	1.4e - 45 to 3.4e 38
Double	8 bytes	4.9e - 324 to 1.8e 308

Floating point numbers are treated as double-precision quantities. To force them to be in single-precision mode, we must append f or F to the numbers. Example: 1.23f, 7.56923F.

Double-precision types are used when we need greater precision in storage of floating point numbers. All mathematical functions, such as sin, cos and sqrt return **double** type values.

Character Type

In order to store character constants in memory, Java provides a character data type called **char**. The char type assumes a size of 2 bytes but basically it can hold only a single character.

Boolean Type

Boolean type is used when we want to test a particular condition during the execution of the programs. There are only 2 values that a Boolean type can take: **true** or **false**. Boolean type is denoted by the keyword **boolean** and used only one bit of storage.

3.4 DECLARATION OF VARIABLES

In java, variables are the names of storage locations. After designing suitable variable names, we must declare them to the compiler. Declaration does 3 things

1. It tells the compiler what the variable name is
2. It specifies what type of data the variable will hold
3. The place of declaration decides the scope of the variable.

A variable must be declared before it is used in the program. A variable can be used to store a value of any data type. The declaration statement defines the type of variable. The general form of declaration of a variable is:

type variable1, variable2,...,variableN;

Variables are separated by commas. A declaration statement must end with a semicolon. Some valid declarations are:

```
int count;  
float x,y;  
double pi;  
byte b;  
char c1,c2,c3;
```

3.5 GIVING VALUES TO VARIABLES

A variable must be given a value after it has been declared but before it is used in an expression. This can be achieved in 2 ways:

1. By using an assignment statement
2. By using a read statement

Assignment Statement

A simple method of giving value to variable is through the assignment statement as follows:

variableName=value;

For example:

```
initialValue=0;  
finalValue=100;  
yes='x';
```

We can also use string assignment expressions as shown below:

```
x=y=z=0;
```

It is also possible to assign a value to a variable at the time of its declaration. This takes the form:

typevariableName=value;

Examples:

```
int finalValue=100;
```

```
char yes='x';
```

```
double total=75.36;
```

The process of giving initial values to variables is known as the **initialization**. The ones that are not initialized are automatically set to zero. The following are valid Java statements:

```
float x,y,z; //declares 3 float variables
```

```
int m=5,n=10; //declares and initializes 2 int variables
```

```
int m,n=10; //declares m and n and initializes n
```

Read Statement

We may also give values to variables interactively through the keyword using the **readLine()** method.

```
import java.io.*;
class Reading
{
    public static void main(String args[]) throws IOException
    {
        DataInputStream in=new DataInputStream(System.in);
        int intNumber=0;
        float floatNumber=0.0f;
        System.out.println("Enter an Integer number:");
        intNumber=Integer.parseInt(in.readLine());
        System.out.println("Enter a float number:");
        floatNumber=Float.parseFloat(in.readLine());
        System.out.println("Integer Number="+intNumber);
        System.out.println("Float Number="+floatNumber);
    }
}
```

OUTPUT

```
Enter an Integer number:
123
Enter a float number:
12.34
Integer Number=:123
Float Number=:12.34
```

The **readLine()** method (which is invoked using an object of the class **DataInputStream**) reads the input from the keyboard as a string which is then CONVERTED TO THE CORRESPONDING DATA TYPE USING THE DATA TYPE WRAPPER CLASSES

3.6 SCOPE OF VARIABLES

Java variables are actually classified into 3 kinds:

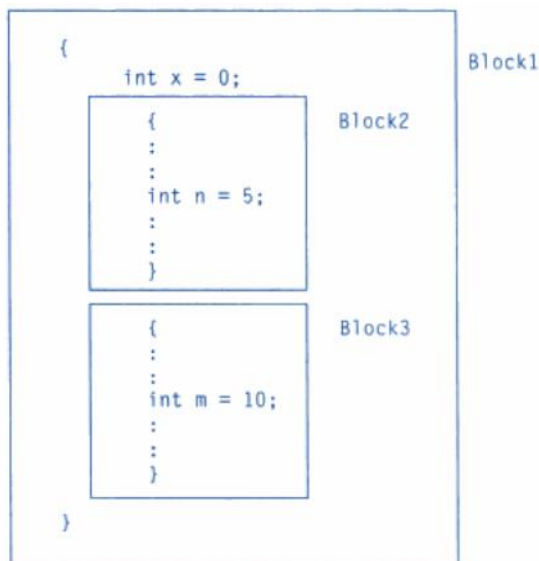
- 1 **instance** variables
- 2 **class** variables
- 3 **local** variables

Instance and **class** variables are declared inside a class. Instance variables are created when the objects are instantiated and therefore they are associated with the objects. They take different values for each object. On the other hand, class variables are global to a class and belong to the entire set of objects that class creates. Only one memory location is created for each class variable.

Variables declared and used inside methods are called **local** variables. They are called so because they are not available for use outside the method definition.

Local variables can also be declared inside program blocks that are defined between an opening brace { and a closing brace }. These variables are visible to the program only from the beginning of its program block to the end of the program block. When the program control leaves a block, all the variables in the block will cease to exist. The area of the program where the variable is accessible (usable) is called its **scope**.

We can have program blocks within other program blocks (called **nesting**) as shown in below figure. Each block can contain its own set of local variable declarations. We cannot, however, declare a variable to have the same name as one in an outer block. In the below figure, the variable **x** declared in Block1 is available in all the 3 blocks. However, the variable **n** declared in Block2 is available only in Block2, because it goes out of the scope at the end of Block2. Similarly, **m** is accessible only in Block3. We cannot declare the variable **x** again in Block2 or Block3.



3.7 SYMBOLIC CONSTANTS

We often use certain unique constants in a program. These constants may appear repeatedly in a number of places in the program. One example of such a constant is 3.142, representing the value of the mathematical constant “pi”. We have 2 problems in this:

1. Problem in modification of the program
2. Problem in understanding the program.

Modifiability

We may like to change the value of “pi” from 3.142 to 3.14159 to improve the accuracy of calculations. In this case, we will have to search throughout the program and explicitly change the value of the constant wherever it has been used. If any value is left unchanged, the program may produce disastrous outputs

Understandability

When a numeric value appears in a program, its use is not always clear, especially when the same value means different things in different places. For example, the number 50 may mean the number of students at one place and the ‘pass marks’ at another place of the same program. We may forget what a certain number meant, when we read the program some days later.

Assignment of a symbolic name to such constants frees us from these problems. For example, we may use the name **STRENGTH** to denote the number of students and **PASS_MARK** to denote the pass marks. Constant values are assigned to these names at the beginning of the program. Subsequent use of the names **STRENGTH** and **PASS_MARK** in the program has the effect of causing their defined values to be automatically substituted at the appropriate points. A constant is declared as follows:

final type symbolic_name=value;

Valid examples are:

```
final int STRENGTH=100;  
final int PASS_MARK=50;  
final float PI=3.24159;
```

Note that:

1. Symbolic names take the same form as variable names. But, they are written in CPITALS to distinguish them from normal variable names. This is only a convention, not a rule
2. After declaration of symbolic constants, they should not be assigned any other value within the program, by using an assignment statement. For example, STRENGTH=200; is illegal

3. They can NOT be declared inside a method. They should be used only as class data members in the beginning of the class.

3.8 TYPE CASTING

We often encounter situations where there is a need to store value of one type into a variable of another type. In such situations, we must cast the value to be stored by proceeding it with the type name in parentheses. The syntax is:

type variable1=(type) variable2;

The process of converting one data type to another is called **casting**. Examples:

```
int m=50;
```

```
byte n=(byte)m;
```

```
long count=(long)m;
```

Four integer types can be cast to any other type except Boolean. Casting into a smaller type may result in a loss of data. Similarly, the float and double can be cast to any other type except Boolean. Casting a floating point value to an integer will result in a loss of the fractional part. The below table lists those casts.

From	To
Byte	short, char, int, long, float, double
Short	int, long, float, double
Char	int, long, float, double
Int	long, float, double
Long	float, double
Float	double

Automatic Conversion

For some types, it is possible to assign a value of one type to a variable of a different type without a cast. Java does the conversion of the assigned value automatically. This is known as automatic type conversion. Automatic type conversion is possible only if the destination type has enough precision to store the source value. For example, int is large enough to hold a byte value. Therefore,

```
byte b=75;
```

```
int a=b;
```

are valid statements

The process of assigning a smaller type to a larger one is known as **widening** or **promotion** and that of assigning a larger type to a smaller one is known as **narrowing**. Note that narrowing may result in loss of information.

```
classTypeWrap  
{
```

```

public static void main(String args[])
{
    System.out.println("Variables Created");
    char c='x';
    byte b=50;
    short s=1996;
    int i=123456789;
    long l=1234567654321L;
    float f1=3.142F;
    float f2=1.2e-5F;
    double d2=0.000000987;
    System.out.println("c="+c);
    System.out.println("b="+b);
    System.out.println("s="+s);
    System.out.println("i="+i);
    System.out.println("l="+l);
    System.out.println("f1="+f1);
    System.out.println("f2="+f2);
    System.out.println("d2="+d2);
    System.out.println("Types Converted");
    short s1=(short)b;
    short s2=(short)i;//produces incorrect result
    float n1=(float)l;
    int m1=(int)f1;//fractional part is lost
    System.out.println("s1="+s1);
    System.out.println("s2="+s2);
    System.out.println("n1="+n1);
    System.out.println("m1="+m1);
}
}

```

OUTPUT

```

Variables Created
c=x
b=50
s=1996
i=123456789
l=1234567654321
f1=3.142
f2=1.2E-5
d2=9.87E-7
Types Converted
s1=50
s2=-13035
n1=1.23456769E12
m1=3

```

Chapter 4

Operators and Expressions

Java supports a rich set of operators. Java operators can be classified into a number of related categories as below:

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators

4. Assignment Operators
5. Increment and Decrement Operators
6. Conditional Operator
7. Special Operators

4.1 ARITHMETIC OPERATORS

Arithmetic operators are used to construct mathematical expressions as in algebra. Java provides all the basic arithmetic operators. They are listed in below table.

Operator	Meaning
+	Addition or unary plus
-	Subtraction or unary minus
*	Multiplication
/	Division
%	Modulo Division (Remainder)

The operators +, -, *, and / all work the same way as they do in other languages. These can operate on any built-in numeric data type of Java. We cannot use these operators on Boolean type. Arithmetic operators are used as shown below:

a-b, a+b, a*b a/b, a%b, -a*b

Here a and b may be variables or constants and are known as operands

Integer Arithmetic

When both the operands in a single arithmetic expression such as a+b are integers, the expression is called an integer expression, and the operation is called integer arithmetic. Integer arithmetic always yields an integer value. If a=14 and b=4 we will have the following results:

a-b=10

a+b=18

a*b=56

a/b=3 (decimal part truncated)

a%b=2 (remainder of integer division)

Real Arithmetic

An arithmetic operation involving only real operands is called real arithmetic. A real operand may assume values either in decimal or exponential notation. The below program shows how arithmetic operators work on floating point values

```
classFloatPoint
{
public static void main(String args[])
{
float a=20.5F,b=6.4F;
System.out.println("a="+a);
System.out.println("b="+b);
```

OUTPUT

a=20.5

b=6.4

a+b=26.9

a-b=14.1

a*b=131.2

a/b=3.203125

a%b=1.2999997

```

System.out.println("a+b="+ (a+b));
System.out.println("a-b="+ (a-b));
System.out.println("a*b="+ (a*b));
System.out.println("a/b="+ (a/b));
System.out.println("a%b="+ (a%b));
}
}

```

Mixed-mode Arithmetic

When one of the operands is real and the other is integer, the expression is called a mixed-mode arithmetic expression. If either operand is of the real type, then the other operand is converted to real and the real arithmetic is performed. The result will be a real. Thus,

15/10.0 produces the result 1.5

Whereas

15/10 produces the results 1

4.2 RELATIONAL OPERATORS

We often compare two quantities. These comparisons can be done with the help of relational operators. The expression containing relational operator is termed as a relational expression. The value of relational expression is either true or false. For example if x=10, then x<20 is true. While 20>x is false.

Java supports 6 relational operators. These operators and their meaning are shown in below table

Operator	Meaning
<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater than or equal to
==	Equal to
!=	Not equal to

A simple relation expression contains only one relational operator and is of the following form:

ae-1 relational operator ae-2

ae-1 and ae-2 are arithmetic expressions, which may be simple constants, variables or combination of them. The below table shows some examples of simple relational expressions and their values.

Expression	Value
4.5<=10	TRUE

4.5 < -10	FALSE
-35 >= 0	FALSE
10 < 7+5	TRUE
a+b==c+d	TRUE

The below program shows the implementation of relational operators

```
class RelationalOperators
{
public static void main(String args[])
{
float a=15.0F, b=20.75F, c=15.0F;
System.out.println("a="+a);
System.out.println("b="+b);
System.out.println("c="+c);
System.out.println("a<b:"+ (a<b));
System.out.println("a>b:"+ (a>b));
System.out.println("a<=c:"+ (a<=c));
System.out.println("a>=b:"+ (a>=b));
System.out.println("a==c:"+ (a==c));
System.out.println("b!=c:"+ (b!=c));
System.out.println("b==a+c:"+ (b==a+c));
}
}
```

OUTPUT

```
a=15.0
b=20.75
c=15.0
a<b:true
a>b:false
a<=c:true
a>=b:false
a==c:true
b!=c:true
b==a+c:false
```

4.3 LOGICAL OPERATORS

Java has 3 logical operators, which are given below

Operator	Meaning
&&	Logical AND
	Logical OR
!	Logical NOT

The logical operators && and || are used when we want to form compound conditions by combining two or more relations. An example is: **a>b && x==10**
An expression of this which combines two or more relational expressions is termed as a logical expression or a compound relational expression. Like the simple relation expressions, a logical expression also yields a value of true or false, according to the truth table shown below. The logical expression given above is true only if both **a>b** and **x==10** are true. If either (or bot) of them are false the expression is false.

Op-1	Op-2	&&	
------	------	----	--

True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

4.4 ASSIGNMENT OPERATORS

Assignment operators are used to assign the value of an expression to variable. The assignment operator is “=”. In addition, Java has a set of ‘shorthand’ assignment operators which are used in the form

v op=exp;

Where **v** is a variable, **exp** is an expression and **op** is a Java binary operator. The **op=** is known as the shorthand assignment operator.

The assignment statement **v op=exp;** is equivalent to **v=v op (exp);**

Consider an example: **x+=y+1;** . This is same as the statement **x=x+(y+1);**

The shorthand operator **+=** means ‘add y+1 to x’. Some of the commonly used shorthand assignment operators are illustrated in the below table

Statement with assignment operator	Statement with shorthand operator
a=a+1	a+=1
a=a-1	a-=1
a=a*(n+1)	a*=n+1
a=a/(n+1)	a/=n+1
a=a%b	a%=b

The use of shorthand assignment operators has 3 advantages:

1. What appears on the left hand side need not be repeated and therefore it becomes easier to write
2. The statement is more concise and easier to read
3. Use of shorthand operator results in a more efficient code

4.5 INCREMENT AND DECREMENT OPERATORS

Java has 2 very useful operators and these are the increment and decrement operators: **++** and **--**

The operator **++** adds 1 to the operand while **--** subtracts 1. Both are unary operators and are used in the following form:

++m; or **m++;**

--m; or **m--;**

++m; is equivalent to **m=m+1;** or **m+=1;**

--m; is equivalent to **m=m-1;** or **m-=1;**

While ++m and m++ mean the same thing when they form statements independently, they behave differently when they are used in expressions on the right-hand side of an assignment statement. Consider the following:

```
m=5;
```

```
y=++m;
```

In this case, the value of y and m would be 6. Suppose, if we rewrite the above statement as

```
m=5;
```

```
y=m++;
```

Then, the value of y would be 5 and m would be 6.

A prefix operator first adds 1 to the operand and then the result is assigned to the variable on left. On the other hand, a postfix operator first assigns the value to the variable on left and then increments the operand. The below program illustrates this

```
class IncDec
{
    public static void main(String args[])
    {
        int m=10, n=20;
        System.out.println("m="+m);
        System.out.println("n="+n);
        System.out.println("++m="+ ++m);
        System.out.println("n++="+ n++);
        System.out.println("m="+m);
        System.out.println("n="+n);
    }
}
```

OUTPUT

```
m=10
n=20
++m=11
n++=20
m=11
n=21
```

4.6 CONDITIONAL OPERATOR

The character pair ?: is a ternary operator available in Java. This operator is used to construct conditional expressions of the form

exp1 ?exp2 : exp3

Where exp1, exp2, exp3 are expressions.

The operator ?: works as follows: exp1 is evaluated first. If it is nonzero (true), then the expression exp2 is evaluated and becomes the value of the conditional expression. Note that only one of the expressions is evaluated. For example:

```
a=10;
```

```
b=15;
```

`x=(a>b) ? a : b;`

In this example, x will be assigned the value of b.

4.7 SPECIAL OPERATORS

Java supports some special operators of interest such as **instanceof** operator and member selection operator (.)

instanceof operator

The **instanceof** is an object reference operator and returns **true** if the object on the left hand side is an instance of the class given on the right hand side. This operator allows us to determine whether the object belongs to a particular class or not.

Example: `person instanceof student` is **true** if the object **person** belongs to the class **student**; otherwise it is **false**.

Dot Operator

The dot operator (.) is used to access the instance variables and methods of class objects. Example:

`person1.age` //reference to the variable age

`person1.salary()` //reference to the method salary()

4.8 MATHEMATICAL FUNCTIONS

Mathematical functions such as cos,sqrt, log etc are frequently used in analysis of real life problems. Java supports these basic math functions through **Math** class defined in the **java.lang** package. The below table lists the math functions define in **Math** class.

Functions	Action
<code>sin(x)</code>	Returns sine of the angle x in radians
<code>cos(x)</code>	Returns cosine of the angle x in radians
<code>tan(x)</code>	Returns tangent of the angle x in radians
<code>asin(y)</code>	Returns the angle whose sine is y
<code>acos(y)</code>	Returns the angle whose cosine is y
<code>atan(y)</code>	Returns the angle whose tangent is y
<code>atan2(x,y)</code>	Returns the angle whose tangent is x/y
<code>pow(x,y)</code>	Returns x raised to y (x^y)
<code>exp(x)</code>	Returns e raised to x (e^x)
<code>log(x)</code>	Returns the natural logarithm of x
<code>sqrt(xd)</code>	Returns the square root of x
<code>ceil(x)</code>	Returns the smallest whole number greater than or equal to x (rounding up)
<code>floor(x)</code>	Returns the largest whole number less than or equal to x (roundeddown)

rint(x)	Returns the truncated value of x
round(x)	Returns the integer closest to the argument
abs(a)	Return the absolute value of a
max(a,b)	Returns the maximum of a and b
min(a,b)	Returns the minimum of a and b

```

class math1
{
public static void main(String args[])
{
double x;
x=Math.max(22,33);
System.out.println("The maximum value:"+x);
x=Math.sqrt(64);
System.out.println("The square root:"+x);
x=Math.abs(-55);
System.out.println("The absolute value:"+x);
x=Math.pow(16,3);
System.out.println("The cube of 16:"+x);
x=Math.min(22,33);
System.out.println("The minimumvalue:"+x);
x=Math.log(3);
System.out.println("The natural log value:"+x);
}
}

```

OUTPUT

```

The maximum value:33.0
The square root:8.0
The absolute value:55.0
The cube of 16:4096.0
The minimumvalue:22.0
The natural log value:1.0986122886681098

```

Chapter 5 Using I/O

5.1 STREAMS

Java programs perform I/O through streams. A **stream** is an abstraction that either produces or consumes information. A stream is linked to a physical device by the Java I/O system. An input stream can abstract many different kinds of input: from a disk file, a keyboard, or a network socket. An output stream may refer to the console, a disk file, or a network connection. Streams are a clean way to deal with input/output.

5.2 BYTE STREAMS AND CHARACTER STREAMS

Java defines two types of streams: byte and character. **Byte streams** provide a convenient means for handling input and output of bytes. Byte streams are used, for example, when reading or writing binary data. **Character streams** provide a convenient means for handling input and output of characters.

The Byte Stream Classes

Byte streams are defined by using two class hierarchies. At the top are two abstract classes: **InputStream** and **OutputStream**. Each of these abstract classes has several concrete subclasses. The byte stream classes in **java.io** are shown in below table. To use the stream classes, you must import **java.io**.

Stream Class	Meaning
BufferedInputStream	Buffered input stream
BufferedOutputStream	Buffered output stream
ByteArrayInputStream	Input stream that reads from a byte array
ByteArrayOutputStream	Output stream that writes to a byte array
DataInputStream	An input stream that contains methods for reading the Java standard data types
DataOutputStream	An output stream that contains methods for writing the Java standard data types
FileInputStream	Input stream that reads from a file
FileOutputStream	Output stream that writes to a file
FilterInputStream	Implements InputStream
FilterOutputStream	Implements OutputStream
InputStream	Abstract class that describes stream input
ObjectInputStream	Input stream for objects
ObjectOutputStream	Output stream for objects
OutputStream	Abstract class that describes stream output
PipedInputStream	Input pipe
PipedOutputStream	Output pipe
PrintStream	Output stream that contains print() and println()
PushbackInputStream	Input stream that allows bytes to be returned to the input stream.
SequenceInputStream	Input stream that is a combination of two or more input streams that will be read sequentially, one after the other

The abstract classes **InputStream** and **OutputStream** define several key methods. Two of the most important are **read()** and **write()**, which, respectively, read and write bytes of data.

The Character Stream Classes

Character streams are defined by using two class hierarchies. At the top are two abstract classes: **Reader** and **Writer**. These abstract classes handle Unicode character streams. Java has several concrete subclasses of each of these. The character stream classes in **java.io** are shown in below table

Stream Class	Meaning
BufferedReader	Buffered input character stream
BufferedWriter	Buffered output character stream
CharArrayReader	Input stream that reads from a character array
CharArrayWriter	Output stream that writes to a character array
FileReader	Input stream that reads from a file
FileWriter	Output stream that writes to a file
FilterReader	Filtered reader
FilterWriter	Filtered writer
InputStreamReader	Input stream that translates bytes to characters
LineNumberReader	Input stream that counts lines
OutputStreamWriter	Output stream that translates characters to bytes
PipedReader	Input pipe
PipedWriter	Output pipe
PrintWriter	Output stream that contains print() and println()
PushbackReader	Input stream that allows characters to be returned to the input stream
Reader	Abstract class that describes character stream input
StringReader	Input stream that reads from a string
StringWriter	Output stream that writes to a string
Writer	Abstract class that describes character stream output

The abstract classes **Reader** and **Writer** define several key methods that the other stream classes implement. Two of the most important methods are **read()** and **write()**, which read and write characters of data, respectively.

5.3 THE PREDEFINED STREAMS

As you know, all Java programs automatically import the **java.lang** package. This package defines a class called **System**, which encapsulates several aspects of the run-time environment. For example, using some of its methods, you can obtain the current time and the settings of various properties associated with the system.

System also contains three predefined stream variables: **in**, **out**, and **err**. These fields are declared as **public**, **static**, and **final** within **System**. This means that they can be used by any other part of your program and without reference to a specific **System** object.

System.out refers to the standard output stream. By default, this is the console.

System.in refers to standard input, which is the keyboard by default.

System.err refers to the standard error stream, which also is the console by default.

System.in is an object of type **InputStream**. **System.out** and **System.err** are objects of type **PrintStream**.

5.4 READING CONSOLE INPUT

In Java, console input is accomplished by reading from **System.in**. To obtain a character-based stream that is attached to the console, wrap **System.in** in a **BufferedReader** object. **BufferedReader** supports a buffered input stream. A commonly used constructor is shown here:

BufferedReader(Reader inputReader)

Here, **inputReader** is the stream that is linked to the instance of **BufferedReader** that is being created. **Reader** is an abstract class. One of its concrete subclasses is

InputStreamReader, which converts bytes to characters. To obtain an **InputStreamReader** object that is linked to **System.in**, use the following constructor:

InputStreamReader(InputStream inputStream)

Because **System.in** refers to an object of type **InputStream**, it can be used for **inputStream**. Putting it all together, the following line of code creates a **BufferedReader** that is connected to the keyboard:

```
BufferedReader br = new BufferedReader(new
                                   InputStreamReader(System.in));
```

After this statement executes, **br** is a character-based stream that is linked to the console through **System.in**.

5.5 READING CHARACTERS

To read a character from a **BufferedReader**, use **read()**. The version of **read()** that we will be using is **int read() throws IOException**

Each time that **read()** is called, it reads a character from the input stream and returns it as an integer value. It returns **-1** when an attempt is made to read at the end of the stream. As you can see, it can throw an **IOException**.

The following program demonstrates **read()** by reading characters from the console until the user types a "q." Notice that any I/O exceptions that might be generated are simply thrown out of **main()**.

```
// Use a BufferedReader to read characters from the console.
import java.io.*;

class BRRead {
    public static void main(String args[]) throws IOException
    {
        char c;
        BufferedReader br = new
            BufferedReader(new InputStreamReader(System.in));
        System.out.println("Enter characters, 'q' to quit.");
        // read characters
        do {
            c = (char) br.read();
            System.out.println(c);
        } while(c != 'q');
    }
}
```

Here is a sample run:

Enter characters, 'q' to quit.

123abcq

1

2

3

a

b

c

q

5.6 READING STRINGS

To read a string from the keyboard, use the version of **readLine()** that is a member of the **BufferedReader** class. Its general form is shown here:

String readLine() throws IOException

The following program demonstrates **BufferedReader** and the **readLine()** method; the program reads and displays lines of text until you enter the word "stop":

```

class BRReadLines {
    public static void main(String args[]) throws IOException
    {
        // create a BufferedReader using System.in
        BufferedReader br = new BufferedReader(new
            InputStreamReader(System.in));

        String str;
        System.out.println("Enter lines of text.");
        System.out.println("Enter 'stop' to quit.");
        do {
            str = br.readLine();
            System.out.println(str);
        } while(!str.equals("stop"));
    }
}

```

Here is a sample run:

```

Enter lines of text.
Enter 'stop' to quit.
This is line one.
This is line two.
Java makes working with strings easy.
Just create String objects.
stop
This is line one.
This is line two.
Java makes working with strings easy.
Just create String objects.

```

5.7 WRITING CONSOLE OUTPUT

Console output is most easily accomplished with **print()** and **println()**

These methods are defined by the class **PrintStream**(which is the type of object referenced by **System.out**).

Because **PrintStream** is an output stream derived from **OutputStream**, it also implements the method **write()**. Thus, **write()** can be used to write to the console. The simplest form of **write()** defined by **PrintStream** is shown here:

void write(int byteval)

This method writes the byte specified by **byteval**. Here is a short example that uses **write()** to output the character "A" followed by a newline to the screen:

```
// Demonstrate System.out.write().
class WriteDemo {
    public static void main(String args[]) {
        int b;

        b = 'A';
        System.out.write(b);
        System.out.write('\n');
    }
}
```

Chapter 6

Decision Making and Branching

When a program breaks the sequential flow and jumps to another part of the code, it is called **branching**. When the branching is based on a particular condition, it is known as **conditional branching**. If branching takes place without any decision, it is known as **unconditional branching**.

Java language processes decision making capabilities and supports the following statements known as control or decision making statements

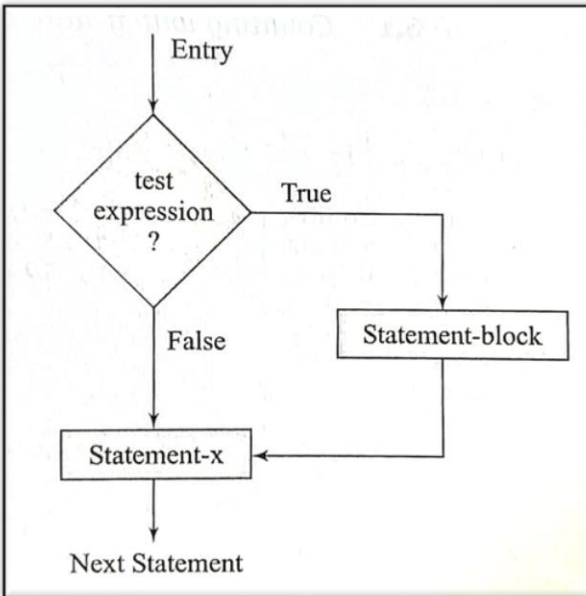
1. **if** statement
2. **switch** statement

6.1 SIMPLE IF STATEMENT

The general form of a simple if statement is

```
if (test expression)
{
    statement-block;
}
statement-x;
```

The ‘statement-block’ may be a single statement or a group of statements. If the **test expression** is true, the **statement-block** will be executed; otherwise the statement-block will be skipped and the execution will jump to the **statement-x**. It should be remembered that when the condition is true both the statement-block and the statement-x are executed in sequence. This is illustrated in below figure.



Consider the following segment of a program that is written for processing of marks obtained in an entrance examination

.....

.....

```

if(category == SPORTS)
{
marks=marks+bonus_marks;
}
System.out.println(marks);

```

.....

.....

The program tests the type of category of the student. If the student belongs to the SPORTS category, then additional bonus_marks are added to his marks before they are printed. For others, bonus_marks are not added.

```

classIfDemo
{
public static void main(String args[])
{
int i=10;
if(i<15)
System.out.println("10 is less than 15");
System.out.println("Outside of if block");
}
}

```

OUTPUT

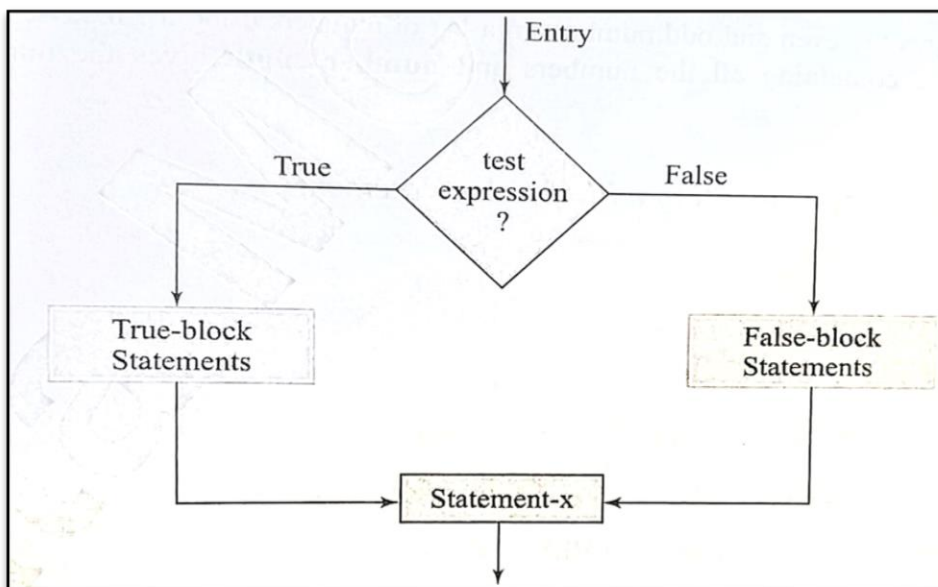
10 is less than 15
Outside of if block

6.2 THE IF..ELSE STATEMENT

The **if...else** statement is an extension of the simple **if** statement. The general form is

```
if(test expression)
{
    true-block statement(s)
}
else
{
    false-block statement(s)
}
statement-x
```

If the **test expression** is true, then the **true-block statement(s)** immediately following the **if** statement are executed; otherwise, the **false-block statement(s)** are executed. In either case, either true-block or false-block will be executed, not both. This is illustrated in below figure. In both the cases, the control is transferred subsequently to the **statement-x**.



Let us consider an example of counting the number of boys and girls in a class

.....

.....

```
if(code == 1)
```

```

        boy=boy+1;
    if(code == 2)
        girl=girl+1;
    .....
    .....

```

The first test determines whether or not the student is a boy. If yes, the number of boys is increased by 1 and the program continues to the second test. The second test again determines whether the student is a girl. This is unnecessary. Once a student is identified as a boy, there is no need to test again for a girl, a student can be either a boy or girl, not both. The above program segment can be modified using the else clause as follows:

```

    .....
    .....
    if(code == 1)
        boy=boy+1;
    else
        girl=girl+1;
    xxx
    .....
    .....

```

Here, if the code is equal to 1, the statement **boy=boy+1;** is executed and the control is transferred to the statement **xxx**, after skipping the **else** part. If the code is not equal to 1, the statement **boy=boy+1;** is skipped and the statement in the **else** part **girl=girl+1;** is executed before the control reaches the statement **xxx**.

```

classIfElseDemo
{
    public static void main(String args[])
    {
        int number=10;
        if(number>0)
        {
            System.out.println("The number is positive");
        }
        else
        {
            System.out.println("The number is not positive");
        }
        System.out.println("Outside of if..else block");
    }
}

```

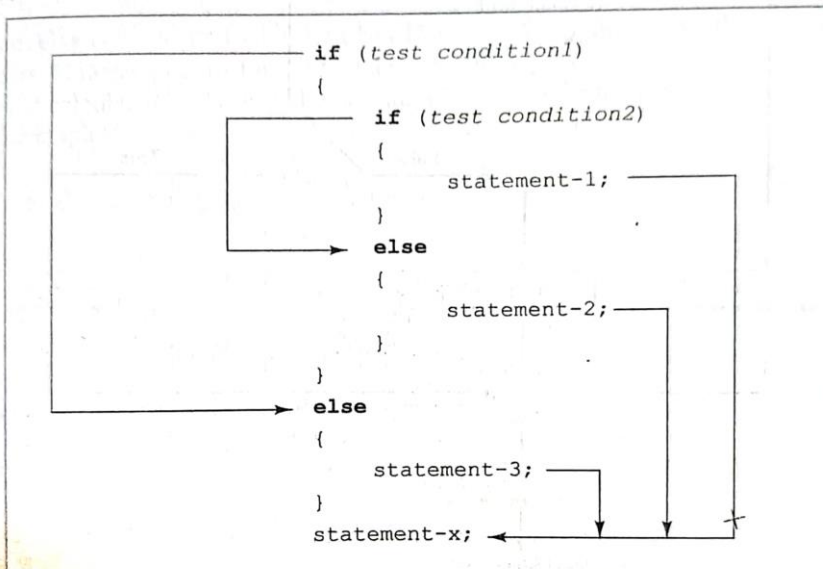
OUTPUT

The number is positive
Outside of if..else block

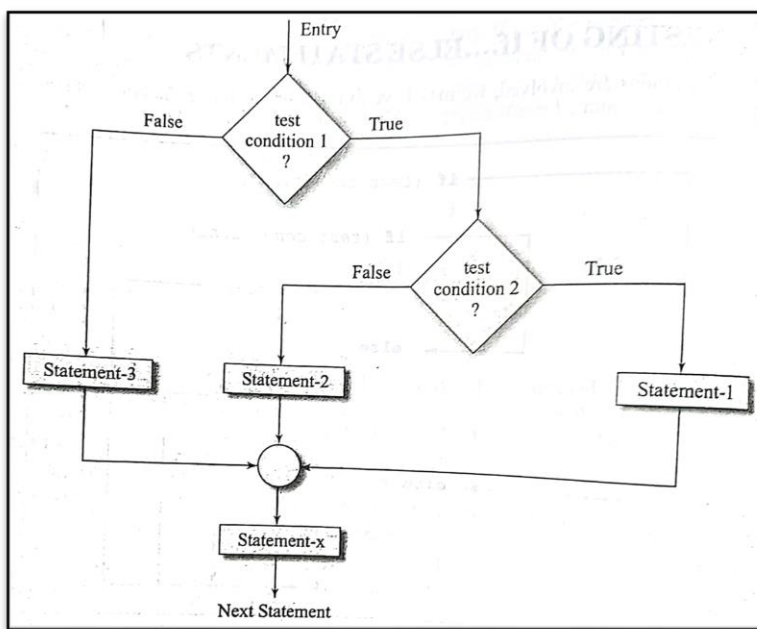
}
}
}

6.3 NESTING OF IF...ELSE STATEMENTS

When a series of decisions are involved, we may have to use more than one if...else statement in nested form as follows:



If the **condition-1** is false, the statement-3 will ne executed; otherwise it continues to perform the second test. If the **condition-2** is true, the statement-1 will be evaluated; otherwise the statemnt-2 will be evaluated and then the control is transferred to the statement-x.



Example:

```

.....
if(sex is female)
{
if(balance>5000)
    bonus=0.05*balance;
else
    bonus=0.02*balance;
}
else
{
    bonus=0.02*balance;
}
balance=balance+bonus;
.....

```

When nesting, care should be exercised to match every **if** with an **else**. Consider the following alternative to the above program

```

if(sex is female)
if(balance> 5000)
    bonus=0.05*balance;
else
    bonus=0.02*balance;
balance=balance+bonus;

```

The following program illustrates nesting of if...else

```

classIfElseNesting
{
    public static void main(String args[])
    {
        int a=325,b=712,c=478;
        System.out.println("Largest value is:");
        if(a>b)
        {
            if(a>c)
            {
                System.out.println(a);
            }
        }
        else
        {

```

OUTPUT

Largest value is:
712

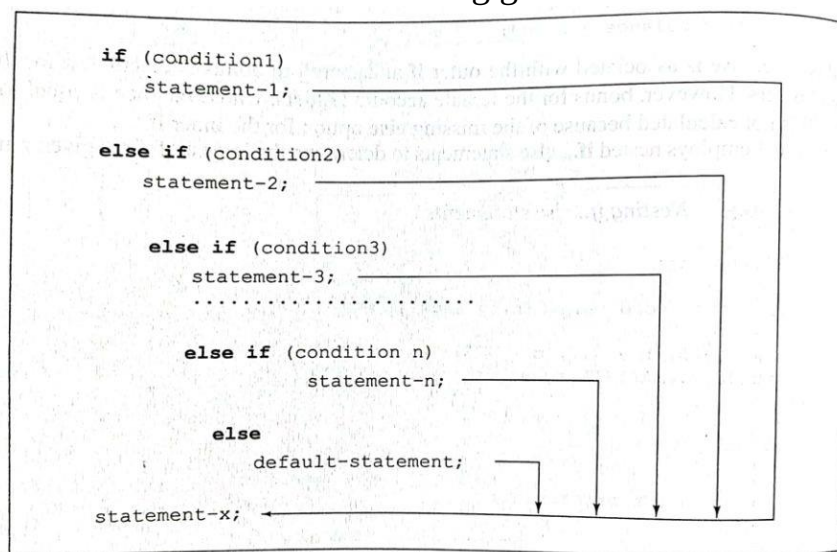
```

System.out.println(c);
}
}
else
{
if(c>b)
{
System.out.println(c);
}
else
{
System.out.println(b);
}
}
}
}
}
}

```

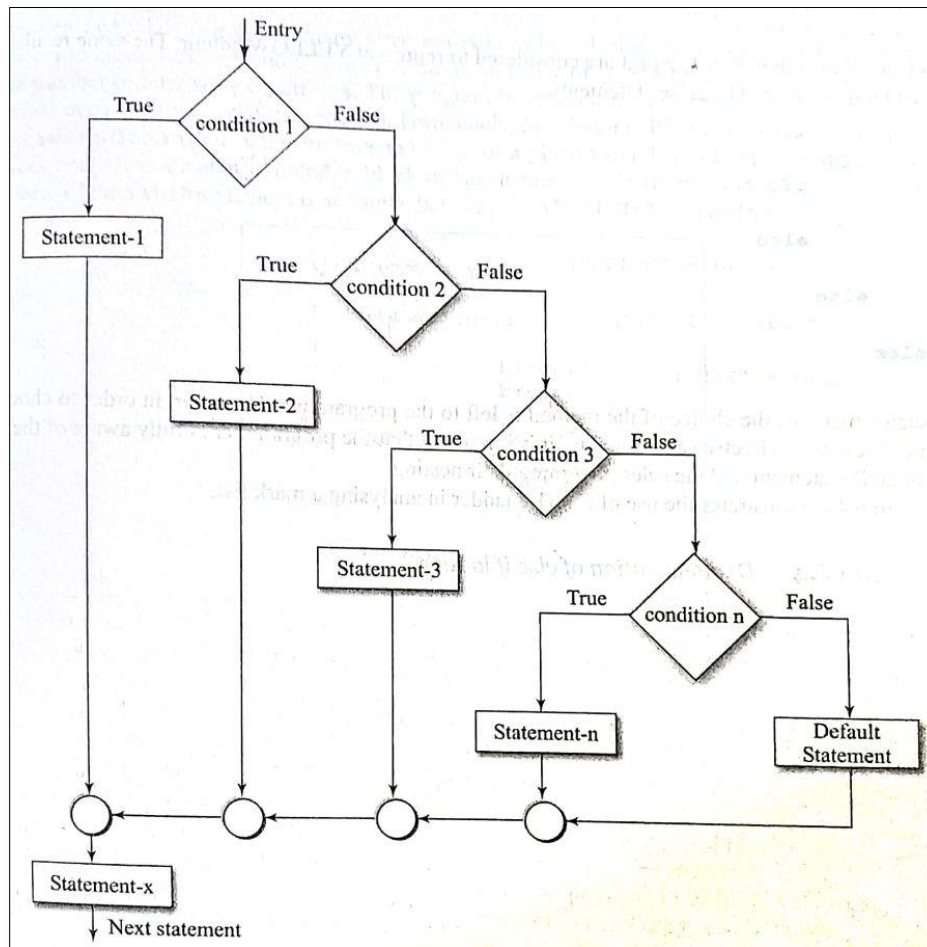
6.4 THE ELSE IF LADDER

There is another way of putting ifs together when multipath decisions are involved. A multipath decision is a chain of ifs in which the statement associated with each **else** is an **if**. It takes the following general form:



This construct is known as the **else if** ladder. The conditions are evaluated from the top (of the ladder), downwards. As soon as the true condition is found, the statement associated with it is executed and the control is transferred to the **statement-x** (skipping the rest of the ladder). When all the n conditions become

false, then the final **else** containing the **default-statement** will be executed. The below figure shows the logic of execution of the **else if** ladder statements.



Let us consider an example of grading the students in an academic institution. The grading is done according to the following rules:

Average Marks	Grade
80 to 100	Honours
60 to 79	First Division
50 to 59	Second Division
40 to 49	Third Division
0 to 39	Fail

This grading can be done using the **else if** ladder as follows:

```

if(marks>79)
    grade="Honours";
elseif(marks>59)
    grade="First Division";
elseif(marks>49)
    grade="Third Division";
else
    grade="Fail";
  
```

```

        grade="Second Division";
    elseif(marks>39)
        grade="Third Division";
    else
        grade="Fail";
System.out.println("Grade:"+grade);

```

The Following code demonstrated else-if-ladder
classElseIfLadder

```

{
public static void main(String args[])
{
int i=20;
if(i==10)
System.out.println("i is 10");
else if(i==15)
System.out.println("i is 15");
else if(i==20)
System.out.println("i is 20");
else
System.out.println("i is not present");
System.out.println("Outside of if-else-if");
}
}

```

OUTPUT

```

i is 20
Outside of if-else-if

```

6.5 THE SWITCH STATEMENT

Java has a built-in multiway decision statement known as **switch**. The **switch** statement tests the value of a given variable (or expression) against a list of **case** values and when a match is found, a block of statements associated with that case is executed. The general form of the **switch** statement is as shown below:

```

switch(expression)
{
    case value-1:
        block-1
        break;
    case value-2:
        block-2
        break;
    .....
}

```

```

.....
default:
    default-block
    break;
}
statement-x;

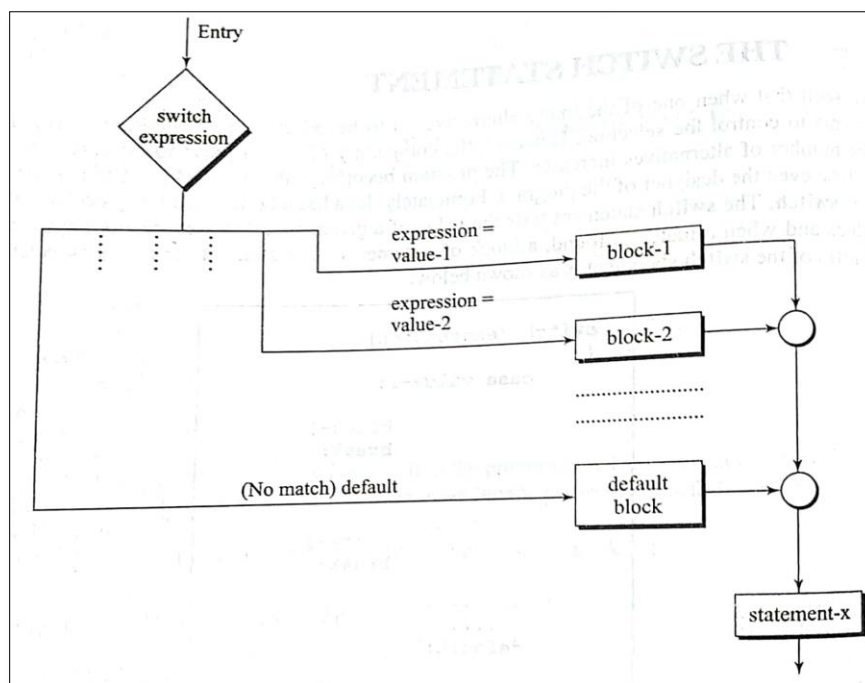
```

The **expression** is an integer expression or characters. **value-1, value-2...** are constants or constant expressions and are known as **case labels**. Each of these values should be unique within a switch statement. **block-1, block2..** are statement lists and may contain zero or more statements. There is no need to put braces around these blocks but it is important to note that case labels end with a colon (:) When the **switch** is executed, the value of the expression is successively compared against the values **value-1, value-2,...** If a case is found whose value matches with the value of the expression, then the block of statements that follow the **case** are executed.

The **break** statement at the end of each block signal the end of a particular case and causes an exit from the **switch** statement, transferring the control to the **statement-x** following the switch.

The **default** is an optional case. When present, it will be executed if the value of the expression does not match with any of the case values. If not present, no action takes place when all matches fail and the control goes to the **statement-x**.

The selection process of **switch** statement is illustrated in the flowchart shown below.



The **switch** statement is illustrated below:

```
classSwitchDemo
{
public static void main(String args[])
{
int day=4;
switch(day)
{
case 1:
System.out.println("Monday");
break;
case 2:
System.out.println("Tuesday");
break;
case 3:
System.out.println("Wednesday");
break;
case 4:
System.out.println("Thursday");
break;
case 5:
System.out.println("Friday");
break;
case 6:
System.out.println("Saturday");
break;
case 7:
System.out.println("Sunday");
break;
}
}
}
```

OUTPUT

Thursday